# C++ Views and Pipelines

NICOLAI M. JOSUTTIS

Cppcon
The C++ Conference

2026
September 12 - 18

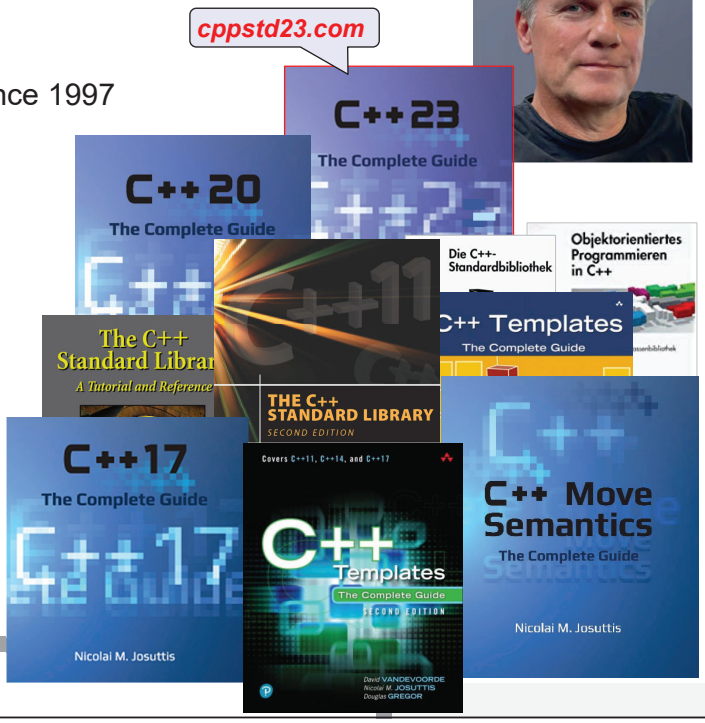
Josuttis C++

## Nicolai M. Josuttis

- **Independent consultant**
  - Continuously learning since 1962
- **C++:**
  - since 1990
  - ISO Standard Committee since 1997
- **Other Topics:**
  - Systems Architect
  - Technical Manager
  - SOA
  - X and OSF/Motif






C++

©2026 by josuttis.com

# C++20

# Views


©2026 by josuttis.com

## Generic `print()` for Containers and Ranges

C++11

```
template <typename T>
void print(const T& coll)
{
  for (const auto& elem : coll) {
    std::cout << elem << ' ';
  }
  std::cout << '\n';
}

std::vector<int> coll1{0, 8, 15, 7, 11, 42, 2};
std::set<int> coll2{0, 8, 15, 7, 11, 42, 2};

print(coll1);
print(coll2);
```

Output:

```
0 8 15 7 11 42 2
0 2 7 8 11 15 42
```


©2026 by josuttis.com

## print() with Class Template Argument Deduction

C++17

```
template <typename T>
void print(const T& coll)
{
  for (const auto& elem : coll) {
    std::cout << elem << ' ';
  }
  std::cout << '\n';
}

std::vector<int> coll1{0, 8, 15, 7, 11, 42, 2};
std::set<int> coll2{0, 8, 15, 7, 11, 42, 2};

print(coll1);
print(coll2);
```

Class Template Argument Deduction deduces `int` as element type (C++17)

Output:

```
0 8 15 7 11 42 2
0 2 7 8 11 15 42
```


©2026 by josuttis.com

## print() with Abbreviated Function Template Syntax

C++20

```
void print(const auto& coll)
{
  for (const auto& elem : coll) {
    std::cout << elem << ' ';
  }
  std::cout << '\n';
}

std::vector<int> coll1{0, 8, 15, 7, 11, 42, 2};
std::set<int> coll2{0, 8, 15, 7, 11, 42, 2} ;

print(coll1);
print(coll2);
```

Templates parameter without specific name

Output:

```
0 8 15 7 11 42 2
0 2 7 8 11 15 42
```


©2026 by josuttis.com

## print() with Range Concepts

C++20

```
void print(const std::ranges::input_range auto& coll)
{
  for (const auto& elem : coll) {
    std::cout << elem << ' ';
  }
  std::cout << '\n';
}
```

Enabled only for ranges with read access

```
std::vector<int> coll1{0, 8, 15, 7, 11, 42, 2};
std::set<int> coll2{0, 8, 15, 7, 11, 42, 2};

print(coll1);
print(coll2);
```

Output:

```
0 8 15 7 11 42 2
0 2 7 8 11 15 42
```


©2026 by josuttis.com

## Views

C++20

```
void print(const std::ranges::input_range auto& coll)
{
  for (const auto& elem : coll) {
    std::cout << elem << ' ';
  }
  std::cout << '\n';
}
```

```
std::vector<int> coll1{0, 8, 15, 7, 11, 42, 2};
std::set<int> coll2{0, 8, 15, 7, 11, 42, 2};

print(coll1);
print(coll2);

print(std::views::take(coll1, 3));        // print first three elements
print(std::views::take(coll2, 3));        // print first three elements

print(coll1 | std::views::take(3));       // print first three elements
print(coll2 | std::views::take(3));       // print first three elements

print(coll2 | std::views::take(3)
            | std::views::transform([](auto v){
                                      return std::to_string(v) + 's';
                                    }));
```

Output:

```
0 8 15 7 11 42 2
0 2 7 8 11 15 42

0 8 15
0 2 7

0 8 15
0 2 7

0s 2s 7s
```


©2026 by josuttis.com

https://www.godbolt.org/z/h1fKPvqTb

## Example of a Pipeline of Views

C++20

```
int main()
{
  std::map<std::string, int> composers {
    {"Bach", 1685},
    {"Beethoven", 1770},
    {"Chopin", 1810},
    {"Mozart", 1756},
    {"Tchaikovsky", 1840},
    {"Vivaldi", 1678},
  };

  // iterate over the names of the first 3 composers since 1700:
  for (const auto& elem : composers
   | std::views::filter([](const auto& y) {
                          return y.second >= 1700;
                        })     // born since 1700
   | std::views::take(3)      // first 3
   | std::views::keys         // names only
  ) {
    std::cout << "- " << elem << '\n';
  }
}
```

Output:

```
- Beethoven
- Chopin
- Mozart
```


©2026 by josuttis.com
https://www.godbolt.org/z/5fYY9hxP4

## Example of a Pipeline of Views

C++20

```
int main()
{
  std::map<std::string, int> composers {
    {"Bach", 1685},
    {"Beethoven", 1770},
    {"Chopin", 1810},
    {"Mozart", 1756},
    {"Tchaikovsky", 1840},
    {"Vivaldi", 1678},
  };

  // iterate over the names of the first 3 composers since 1700:
  auto namesOfFirst3Since1700 = composers
   | std::views::filter([](const auto& y) {
       return y.second >= 1700;
     })                       // born since 1700
   | std::views::take(3)      // first 3
   | std::views::keys;        // names only

  for (const auto& elem : namesOfFirst3Since1700) {
    std::cout << "- " << elem << '\n';
  }
}
```

Output:

```
- Beethoven
- Chopin
- Mozart
```


©2026 by josuttis.com
https://www.godbolt.org/z/sa7PPP6r9

## Example of a Pipeline of Views

C++20

```
int main()
{
  std::map<std::string, int> composers {
    {"Bach", 1685},
    {"Beethoven", 1770},
    {"Chopin", 1810},
    {"Mozart", 1756},
    {"Tchaikovsky", 1840},
    {"Vivaldi", 1678},
  };

  // the first 3 since 1700:
  auto first3Since1700 = std::views::filter([](const auto& y) {
                                              return y.second >= 1700;
                                            })          // born since 1700
                       | std::views::take(3);         // first 3

  // iterate over the names of the first 3 composers since 1700:
  for (const auto& elem : composers | first3Since1700 | std::views::keys) {
    std::cout << "- " << elem << '\n';
  }
}
```

Output:

```
- Beethoven
- Chopin
- Mozart
```


©2026 by josuttis.com
https://www.godbolt.org/z/WzPfe3e83

## Using a Pipeline to Modify Elements

C++20

```
std::vector<int> values{1, 2, 3, 4, 5, 6, 7, 8};
print(values);

// square first five values:
for (int& i : values | std::views::take(5)) {
  i = i * i;
}
print(values);

// square even values:
auto isEven = [] (const auto& val) { return val % 2 == 0; };
for (int& i : values | std::views::filter(isEven)) {
  i = i * i;
}
print(values);
```

Output:

```
1 2 3 4 5 6 7 8
1 4 9 16 25 6 7 8
1 16 9 256 25 36 7 64
```


©2026 by josuttis.com
https://www.godbolt.org/z/We45bY3sd

## A Pipeline of Views with Generating View

C++20

```
bool isPrime(int);   // forward declaration

auto asMilliseconds = [] (auto val) {
  return std::chrono::milliseconds{val};
};

// view of 4th to 11th prime numbers as milliseconds:
auto v = std::views::iota(1)                          // generates 1, 2, 3, ...
         | std::views::filter(isPrime)                // only prime numbers
         | std::views::drop(3)                        // skip first 3
         | std::views::take(8)                        // take next 8
         | std::views::transform(asMilliseconds);     // as milliseconds

for (const auto& elem : v) {
  std::cout << elem << '\n';
}
```

Output:

```
7ms
11ms
13ms
17ms
...
31ms
```


©2026 by josuttis.com
https://www.godbolt.org/z/aYGz8ba8E

## A Pipeline of Views with Generating View

C++20

```
bool isPrime(int);   // forward declaration

auto asMilliseconds = [] (auto val) {
  return std::chrono::milliseconds{val};
};

// view of 4th to 11th prime numbers as milliseconds:
auto v = std::views::iota(1)                          // generates 1, 2, 3, ...
         | std::views::filter(isPrime)                // only prime numbers
         | std::views::drop(3)                        // skip first 3
         | std::views::take(8)                        // take next 8
         | std::views::transform(asMilliseconds);     // as milliseconds

for (const auto& elem : v) {
  std::cout <<
}
```

Output:

**Type of v is:**

```
std::ranges::transform_view<
  std::ranges::take_view<
    std::ranges::drop_view<
      std::ranges::filter_view<
        std::ranges::iota_view<int, std::unreachable_sentinel_t>,
        bool(*)(int)>>>,
  typeOfAsMilliseconds>
```


©2026 by josuttis.com
https://www.godbolt.org/z/aYGz8ba8E

## Combining Views

C++23

```
std::vector<std::string> coll{"tic", "tac", "toe"};

// print elements with their index:
int i = 0;
for (const auto& name : coll) {
  std::cout << i++ << ": " << name << '\n';
}

// print elements with their index:
for (auto [i, name] : coll | std::views::enumerate) {
  std::cout << i << ": " << name << '\n';
}

// print elements with their position (starting with 1):
for (auto [i, name] : std::views::zip(std::views::iota(1), coll)) {
  std::cout << i << ": " << name << '\n';
}
```

Output:

```
0: tic
1: tac
2: toe

0: tic
1: tac
2: toe

1: tic
2: tac
3: toe
```


©2026 by josuttis.com
https://www.godbolt.org/z/qccWdEbvx

## Sorting Multiple Ranges with the Zip View

C++23

```
int main()
{
  std::vector<std::string> first{"Brad", "Tina", "James"};
  std::deque<std::string> last{"Pitt", "Turner", "Bond"};

  std::println("first: {}", first);
  std::println("last:  {}\n", last);

  // use zip view to sort two ranges according to second range:
  std::ranges::sort(std::views::zip(first, last),          // let elems refer to both ranges
                    [] (auto tpl1, auto tpl2) {            // compare acc. to 2nd range
                      return std::get<1>(tpl1) < std::get<1>(tpl2);
                    });

  std::println("first: {}", first);
  std::println("last:  {}", last);
}
```

Output:

```
first: ["Brad", "Tina", "James"]
last:  ["Pitt", "Turner", "Bond"]

first: ["James", "Brad", "Tina"]
last:  ["Bond", "Pitt", "Turner"]
```


©2026 by josuttis.com
https://www.godbolt.org/z/z7G895a8o

## Parallel Processing with Views

C++23

```
void processParallel(auto& coll, auto op)   // call op for all elements
{
  auto foreach = [] (auto& subView, auto op) {
    for (auto&& elem : subView) {
      op(elem);
    }
  };

  auto v1 = coll | std::views::stride(3);   // every third elem
  auto v2 = coll | std::views::drop(1)      // every third elem from 2nd
                 | std::views::stride(3);
  auto v3 = coll | std::views::drop(2)      // every third elem from 3rd
                 | std::views::stride(3);

  // process data in 3 parallel threads:
  std::jthread t1{foreach, v1, op};         // call op for every third elem
  std::jthread t2{foreach, v2, op};         // call op for every third elem from 2nd
  std::jthread t3{foreach, v3, op};         // call op for every third elem from 3rd

  t1.join();
  t2.join();
  t3.join();
}
```

generic helper to call op for each element

```
auto square = [] (auto& elem) {
  elem *= elem;
};
processParallel(data, square);
```


©2026 by josuttis.com
https://www.godbolt.org/z/eGxsWcaMs

## C++ Views and Pipelines

## How Views Operate


©2026 by josuttis.com

## How Views Operate

C++20

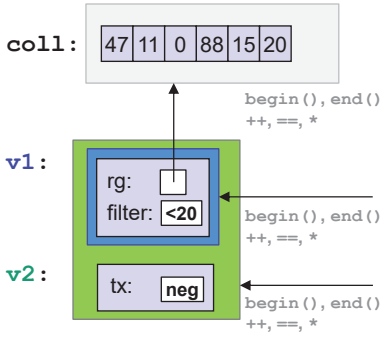
```
std::vector<int> coll{47, 11, 0, 88, 15, 20};
print(coll);

auto less20 = [] (auto val) { return val < 20; };
print(coll | std::views::filter(less20)
           | std::views::transform(std::negate{}));
```

```
auto v1 = std::views::filter(coll, less20);
auto v2 = std::views::transform(v1, std::negate{});
```

Output:

```
47 11 0 88 15 20
-11 0 -15
```





©2026 by josuttis.com

## How Views Operate

C++20

```
std::vector<int> coll{47, 11, 0, 88, 15, 20};
print(coll);

auto less20 = [] (auto val) { return val < 20; };
print(coll | std::views::filter(less20)
           | std::views::transform(std::negate{}));
```

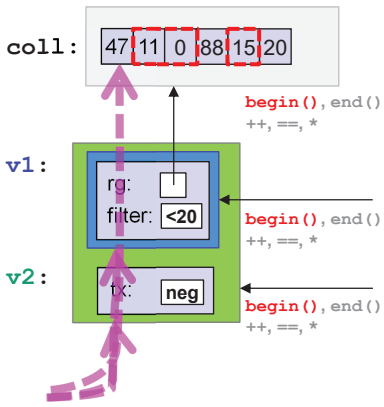
```
auto v1 = std::views::filter(coll, less20);
auto v2 = std::views::transform(v1, std::negate{});
```

print() step by step:

```
v2.begin();
```

Output:

```
47 11 0 88 15 20
-11 0 -15
```





©2026 by josuttis.com

## How Views Operate

C++20

```
std::vector<int> coll{47, 11, 0, 88, 15, 20};
print(coll);

auto less20 = [] (auto val) { return val < 20; };
print(coll | std::views::filter(less20)
           | std::views::transform(std::negate{}));
```

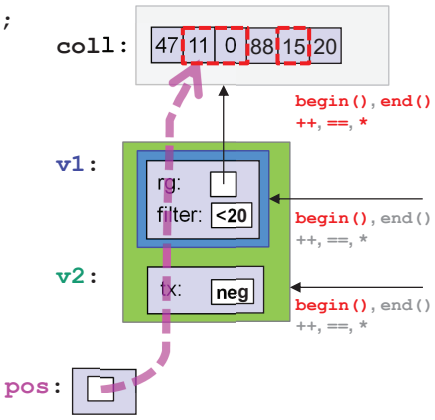
```
auto v1 = std::views::filter(coll, less20);
auto v2 = std::views::transform(v1, std::negate{});
```

**print() step by step:**

```
auto pos = v2.begin();
```

Output:

```
47 11 0 88 15 20
-11 0 -15
```





©2026 by josuttis.com

## How Views Operate

C++20

```
std::vector<int> coll{47, 11, 0, 88, 15, 20};
print(coll);

auto less20 = [] (auto val) { return val < 20; };
print(coll | std::views::filter(less20)
           | std::views::transform(std::negate{}));
```

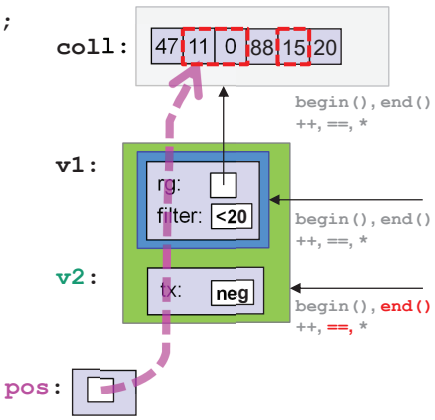
```
auto v1 = std::views::filter(coll, less20);
auto v2 = std::views::transform(v1, std::negate{});
```

**print() step by step:**

```
auto pos = v2.begin();
if (pos != v2.end())
```

Output:

```
47 11 0 88 15 20
-11 0 -15
```





©2026 by josuttis.com

## How Views Operate

C++20

```
std::vector<int> coll{47, 11, 0, 88, 15, 20};
print(coll);

auto less20 = [] (auto val) { return val < 20; };
print(coll | std::views::filter(less20)
           | std::views::transform(std::negate{}));
```

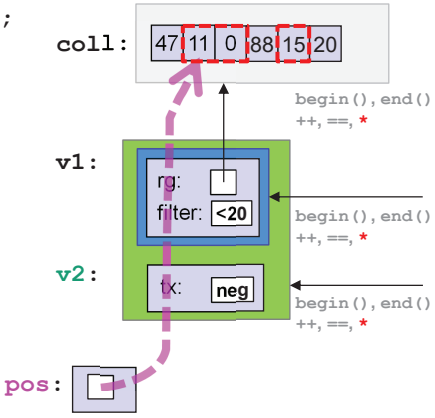
```
auto v1 = std::views::filter(coll, less20);
auto v2 = std::views::transform(v1, std::negate{});
```

**print() step by step:**

```
auto pos = v2.begin();
if (pos != v2.end())
  std::cout << *pos << '\n';
```

Output:

```
47 11 0 88 15 20
-11 0 -15
-11
```





©2026 by josuttis.com

## How Views Operate

C++20

```
std::vector<int> coll{47, 11, 0, 88, 15, 20};
print(coll);

auto less20 = [] (auto val) { return val < 20; };
print(coll | std::views::filter(less20)
           | std::views::transform(std::negate{}));
```

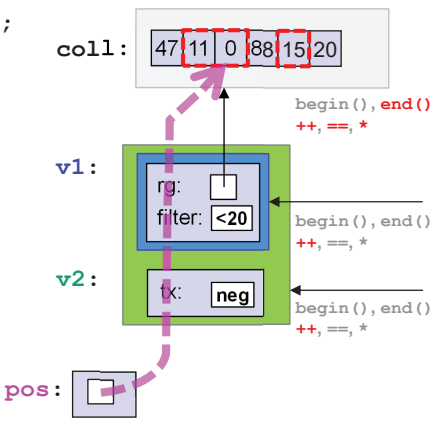
```
auto v1 = std::views::filter(coll, less20);
auto v2 = std::views::transform(v1, std::negate{});
```

**print() step by step:**

```
auto pos = v2.begin();
if (pos != v2.end())
  std::cout << *pos << '\n';
  ++pos;
```

Output:

```
47 11 0 88 15 20
-11 0 -15
-11
```





©2026 by josuttis.com

## How Views Operate

C++20

```
std::vector<int> coll{47, 11, 0, 88, 15, 20};
print(coll);

auto less20 = [] (auto val) { return val < 20; };
print(coll | std::views::filter(less20)
           | std::views::transform(std::negate{}));
```

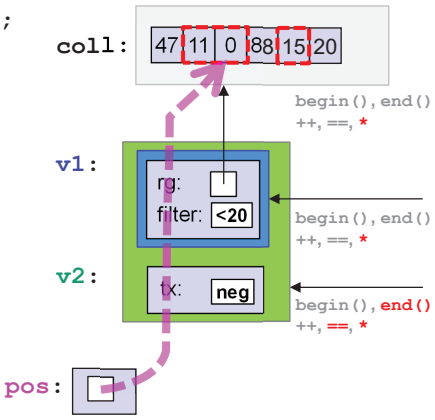
```
auto v1 = std::views::filter(coll, less20);
auto v2 = std::views::transform(v1, std::negate{});
```

**print() step by step:**

```
auto pos = v2.begin();
if (pos != v2.end())
  std::cout << *pos << '\n';
  ++pos;
if (pos != v2.end())
  std::cout << *pos << '\n';
```

**Output:**

```
47 11 0 88 15 20
-11 0 -15
-11
0
```






## How Views Operate

C++20

```
std::vector<int> coll{47, 11, 0, 88, 15, 20};
print(coll);

auto less20 = [] (auto val) { return val < 20; };
print(coll | std::views::filter(less20)
           | std::views::transform(std::negate{}));
```

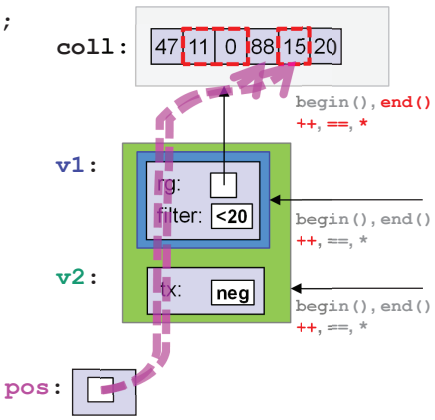
```
auto v1 = std::views::filter(coll, less20);
auto v2 = std::views::transform(v1, std::negate{});
```

**print() step by step:**

```
auto pos = v2.begin();
if (pos != v2.end())
  std::cout << *pos << '\n';
  ++pos;
if (pos != v2.end())
  std::cout << *pos << '\n';
  ++pos;
```

**Output:**

```
47 11 0 88 15 20
-11 0 -15
-11
0
```



## How Views Operate

C++20

```
std::vector<int> coll{47, 11, 0, 88, 15, 20};
print(coll);

auto less20 = [] (auto val) { return val < 20; };
print(coll | std::views::filter(less20)
           | std::views::transform(std::negate{}));
```

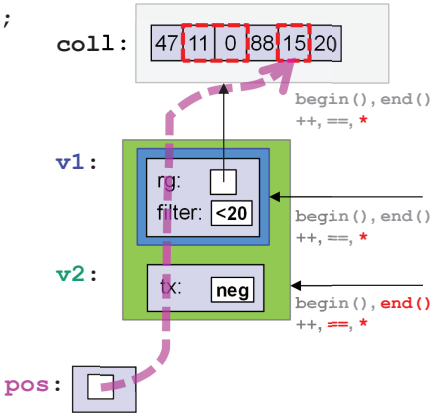
```
auto v1 = std::views::filter(coll, less20);
auto v2 = std::views::transform(v1, std::negate{});
```

**print() step by step:**

```
auto pos = v2.begin();
if (pos != v2.end())
  std::cout << *pos << '\n';
  ++pos;
if (pos != v2.end())
  std::cout << *pos << '\n';
  ++pos;
if (pos != v2.end())
  std::cout << *pos << '\n';
```

Output:

```
47 11 0 88 15 20
-11 0 -15
-11
0
-15
```






## How Views Operate

C++20

```
std::vector<int> coll{47, 11, 0, 88, 15, 20};
print(coll);

auto less20 = [] (auto val) { return val < 20; };
print(coll | std::views::filter(less20)
           | std::views::transform(std::negate{}));
```

```
auto v1 = std::views::filter(coll, less20);
auto v2 = std::views::transform(v1, std::negate{});
```

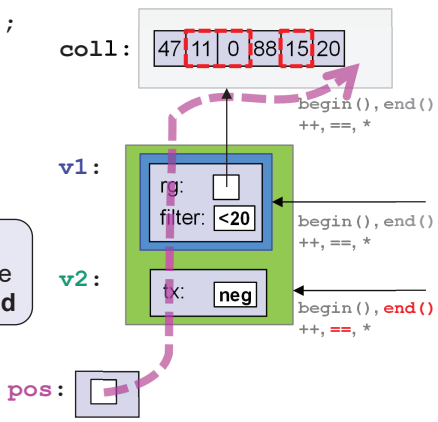
**print() step by step:**

```
auto pos = v2.begin();
if (pos != v2.end())
  std::cout << *pos << '\n';
  ++pos;
if (pos != v2.end())
  std::cout << *pos << '\n';
  ++pos;
if (pos != v2.end())
  std::cout << *pos << '\n';
  ++pos;
if (pos != v2.end())
```

***Pull model***:
Next element and its value are processed **on demand**

Output:

```
47 11 0 88 15 20
-11 0 -15
-11
0
-15
```



## Exact Type of Views

C++20

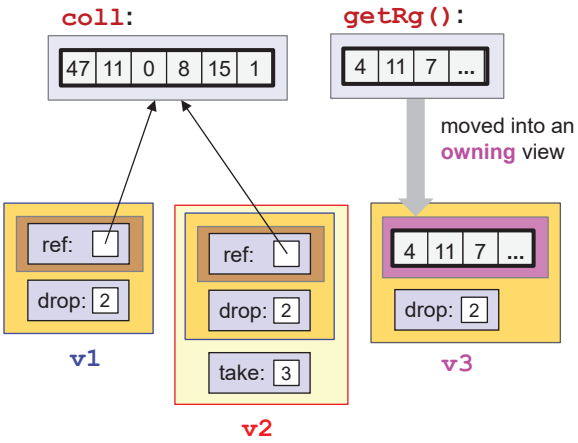
- For named ranges, views create a ref view
- For views, views create a local copy
- For temporary ranges, views create an owning view
  - move-initialized with the range

```
std::vector<int> coll{47, 11, 0, 8, 15, 1};

auto v1 = coll | std::views::drop(2);
     drop_view<ref_view<vector<int>>>

auto v2 = v1 | std::views::take(3);
     take_view<drop_view<ref_view<vector<int>>>>


auto getRg()
{
  return std::vector{4, 11, 7, ...};
}
auto v3 = getRg() | std::views::drop(2);
     drop_view<owning_view<vector<int>>>
```





©2026 by josuttis.com
https://www.godbolt.org/z/ro139nq9W

## Parallel Processing with Views

C++23

```
void processParallel(auto& coll, auto op)   // call op for all elements
{
  auto foreach = [] (auto& subView, auto op) {
    for (auto&& elem : subView) {
      op(elem);
    }
  };

  auto v1 = coll | std::views::stride(3);          // every third elem
  auto v2 = coll | std::views::drop(1)             // every third elem from 2nd
                 | std::views::stride(3);
  auto v3 = coll | std::views::drop(2)             // every third elem from 3rd
                 | std::views::stride(3);

  // process data in 3 parallel threads:
  std::jthread t1{foreach, v1, op};                // call op for every third elem
  std::jthread t2{foreach, v2, op};                // call op for every third elem from 2nd
  std::jthread t3{foreach, v3, op};                // call op for every third elem from 3rd

  t1.join();
  t2.join();
  t3.join();
}
```

generic helper to call op for each element

```
auto square = [] (auto& elem) {
  elem *= elem;
};
processParallel(data, square);
```


©2026 by josuttis.com
https://www.godbolt.org/z/eGxsWcaMs

## Parallel Processing with Views

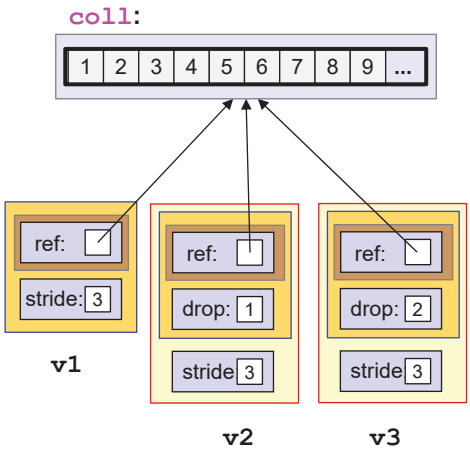
C++23

```
void processParallel(auto& coll, auto op)
{
  auto foreach = [] (auto& subView, auto op) {
    for (auto&& elem : subView) {
      op(elem);
    }
  };

  auto v1 = coll | std::views::stride(3);
  auto v2 = coll | std::views::drop(1)
                 | std::views::stride(3);
  auto v3 = coll | std::views::drop(2)
                 | std::views::stride(3);

  // process data in 3 parallel threads:
  std::jthread t1{foreach, v1, op};
  std::jthread t2{foreach, v2, op};
  std::jthread t3{foreach, v3, op};

  t1.join();
  t2.join();
  t3.join();
}
```






## C++ Views and Pipelines

## Issues with C++ Views

## Views Considered Harmful

C++20

- There are pitfalls and traps
  - Conceptional
    - Something that lies in the principle of ranges and pipeline
  - Intentional
    - Flexibility over simplicity
  - Accidental
- Different opinions
  - "Feature" versus "Design mistake"
- Backward compatibility makes it hard to fix something (now)


©2026 by josuttis.com

## Views

C++20/C++23

- Differ from containers:
  - Reference semantics
  - `begin()` and `end()` might have different types
  - Caching
  - Propagation of `const`
- Breaking of basic programming patterns:
  - `empty()` check might change program behavior
  - Printing elements might change program behavior
  - `const` might not work

Beware of:

- Caching views
- Const Correctness
- The Filter View Fiasco


©2026 by josuttis.com

# C++ Views and Pipelines

## Invalid Pipelines


©2026 by josuttis.com

## A Pipeline of Views with Generating View

C++20

```
bool isPrime(int);   // forward declaration

auto asMilliseconds = [] (auto val) {
  return std::chrono::milliseconds{val};
};

// view of 4th to 11th prime numbers as milliseconds:
auto v = std::views::iota(1, 100)                  // generates 1, 2, ... 99
         | std::views::filter(isPrime)             // only prime numbers
         | std::views::drop(3)                     // skip first 3
         | std::views::take(8)                     // take next 8
         | std::views::transform(asMilliseconds);  // as milliseconds

for (const auto& elem : v) {
  std::cout << elem << '\n';
}
```

Output:

```
7ms
11ms
13ms
17ms
...
31ms
```


©2026 by josuttis.com
https://www.godbolt.org/z/aYGz8ba8E

## Using Big Pipelines

C++20

```
// view of 4th to 11th prime numbers as milliseconds:
auto v = std::views::iota(1, 100)                    // generates 1, 2, ... 99
         | std::views::filter(isPrime)               // only prime numbers
         | std::views::drop(3)                       // skip first 3
         | std::views::take(8)                       // take next 8
         | std::views::transform(asMilliseconds);    // as milliseconds
```

```
for (const auto& elem : v) {                         // OK
  std::cout << elem << '\n';
}
```

```
std::vector coll{v.begin(), v.end()};                // ERROR
```


©2026 by josuttis.com
https://www.godbolt.org/z/1jbfd8jfK

## Using Big Pipelines

C++20

```
// view of 4th to 11th prime numbers as milliseconds:
auto v = std::views::iota(1, 100)                    // generates 1, 2, ... 99
         | std::views::filter(isPrime)               // only prime numbers
         | std::views::drop(3)                       // skip first 3

         | std::views::transform(asMilliseconds);    // as milliseconds
```

```
for (const auto& elem : v) {                         // OK
  std::cout << elem << '\n';
}
```

```
std::vector coll{v.begin(), v.end()};                // OK
```


©2026 by josuttis.com

## Using Big Pipelines

C++20

```
// view of 4th to 11th prime numbers as milliseconds:
auto v = std::views::iota(1, 100)                    // generates 1, 2, ... 99

         | std::views::drop(3)                       // skip first 3
         | std::views::take(8)                       // take next 8
         | std::views::transform(asMilliseconds);    // as milliseconds
```

```
for (const auto& elem : v) {                         // OK
  std::cout << elem << '\n';
}
```

```
std::vector coll{v.begin(), v.end()};                // OK
```


©2026 by josuttis.com

## Using Big Pipelines

C++20

```
// view of 4th to 11th prime numbers as milliseconds:
auto v = std::views::iota(1)                         // generates 1, 2, ...

         | std::views::drop(3)                       // skip first 3
         | std::views::take(8)                       // take next 8
         | std::views::transform(asMilliseconds);    // as milliseconds
```

```
for (const auto& elem : v) {                         // OK
  std::cout << elem << '\n';
}
```

```
std::vector coll{v.begin(), v.end()};                // ERROR
```


©2026 by josuttis.com

## Using Big Pipelines

C++20

```
// view of 4th to 11th prime numbers as milliseconds:
auto v = std::views::iota(1, 100)                    // generates 1, 2, ... 99
         | std::views::filter(isPrime)               // only prime numbers
         | std::views::drop(3)                       // skip first 3
         | std::views::take(8)                       // take next 8
         | std::views::transform(asMilliseconds);    // as milliseconds
```

```
for (const auto& elem : v) {                         // OK
  std::cout << elem << '\n';
}
```

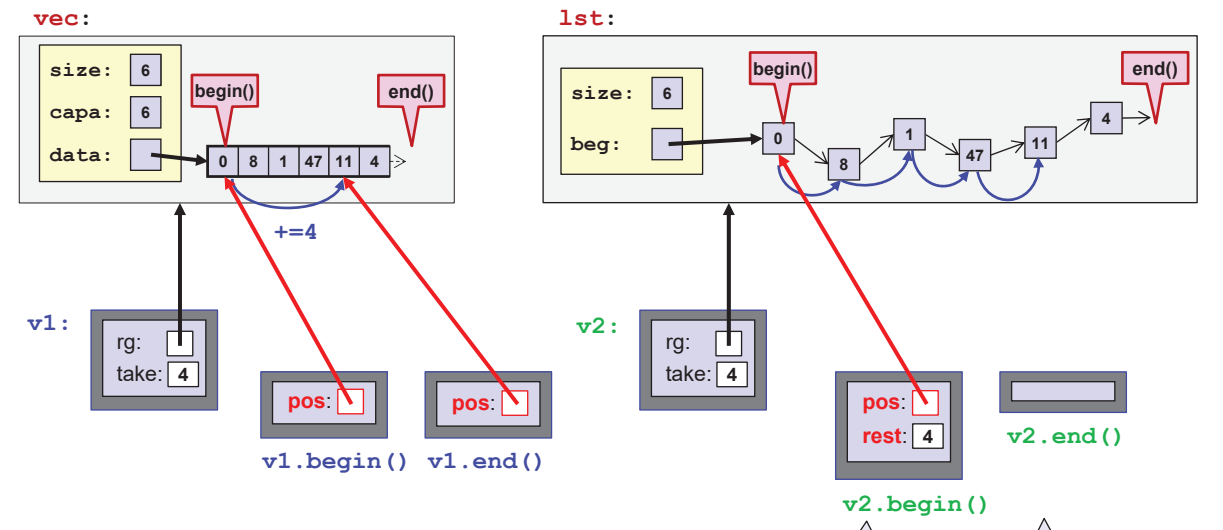
```
std::vector coll{v.begin(), v.end()};                // ERROR
```

For this vector constructor,
**begin and end must have the same type**
(must be a *common range*)



## Why Are Views Not Always Common?

C++20




```
std::vector<int> vec{0, 8, 1, 47, 11, 4};
auto v1 = vec | std::views::take(4);
std::list<int> lst{0, 8, 1, 47, 11, 4};
auto v2 = lst | std::views::take(4);
```

Type: `std::counted_iterator<>`

Type: `std::default_sentinel`

## Using Big Pipelines

C++20

```
// view of 4th to 11th prime numbers as milliseconds:
auto v = std::views::iota(1, 100)                    // generates 1, 2, ... 99
         | std::views::filter(isPrime)               // only prime numbers
         | std::views::drop(3)                       // skip first 3
         | std::views::take(8)                       // take next 8
         | std::views::transform(asMilliseconds);    // as milliseconds
```

random-access common view

bidirectional common view

no longer common (due to bidirectional)

```
for (const auto& elem : v) {                         // OK
  std::cout << elem << '\n';
}

std::vector coll{v.begin(), v.end()};                // ERROR
```



## Using Big Pipelines

C++20

```
// view of 4th to 11th prime numbers as milliseconds:
auto v = std::views::iota(1, 100)                    // generates 1, 2, ... 99
         | std::views::filter(isPrime)               // only prime numbers
         | std::views::drop(3)                       // skip first 3
         | std::views::take(8)                       // take next 8
         | std::views::transform(asMilliseconds);    // as milliseconds
```

random-access common view

bidirectional common view

no longer common (due to bidirectional)

```
for (const auto& elem : v) {                         // OK
  std::cout << elem << '\n';
}

std::vector coll{v.begin(), v.end()};                // ERROR

auto vc = v | std::views::common;                    // harmonize begin and end
std::vector coll{vc.begin(), vc.end()};              // store in vector
```

## Using Big Pipelines

C++23

```
// view of 4th to 11th prime numbers as milliseconds:
auto v = std::views::iota(1, 100)                    // generates 1, 2, ... 99
         | std::views::filter(isPrime)               // only prime numbers
         | std::views::drop(3)                       // skip first 3
         | std::views::take(8)                       // take next 8
         | std::views::transform(asMilliseconds);    // as milliseconds
```

random-access common view

bidirectional common view

no longer common (due to bidirectional)

```
for (const auto& elem : v) {                         // OK
  std::cout << elem << '\n';
}

std::vector coll{v.begin(), v.end()};                // ERROR

auto vc = v | std::views::common;                    // harmonize begin and end
std::vector coll{vc.begin(), vc.end()};              // store in vector

auto coll1 = v | std::ranges::to<std::vector>();     // store in vector (C++23)

std::vector coll2{std::from_range, v};               // store in vector (C++23)
```


©2026 by josuttis.com
https://www.godbolt.org/z/dvnWdb455

## C++ Views and Pipelines

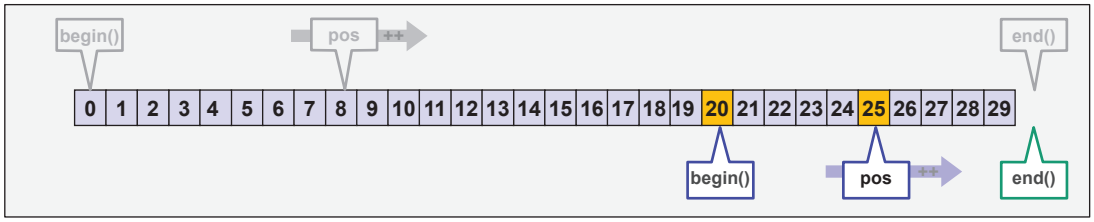
## Caching Views


©2026 by josuttis.com

## Iteration with a Filter View

C++20

```
v = coll | std::views::filter(someAtEnd)
```




- **Manual iteration:**

```
for (auto pos = rg.begin(); pos != rg.end(); ++pos) {
  process(*pos);
}
```

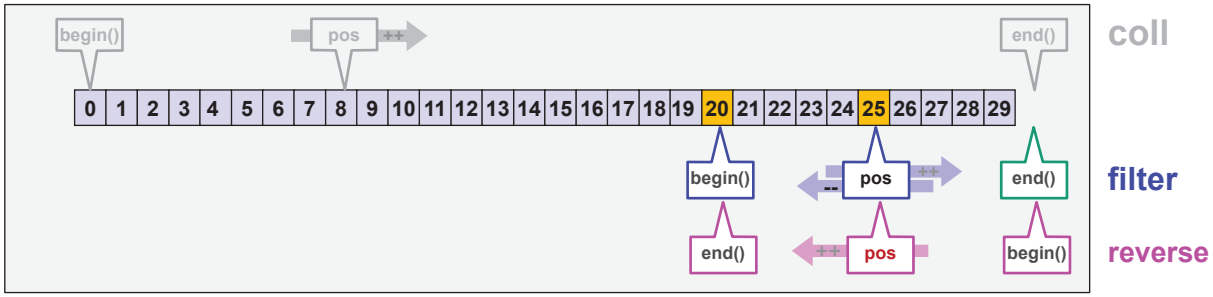
One call of **filter begin()**
Multiple calls of **filter end()**


©2026 by josuttis.com

## Reverse Iteration with a Filter View

C++20

```
v = coll | std::views::filter(someAtEnd) | std::views::reverse
```




- **Manual iteration:**

```
for (auto pos = rg.begin(); pos != rg.end(); ++pos) {
  process(*pos);
}
```

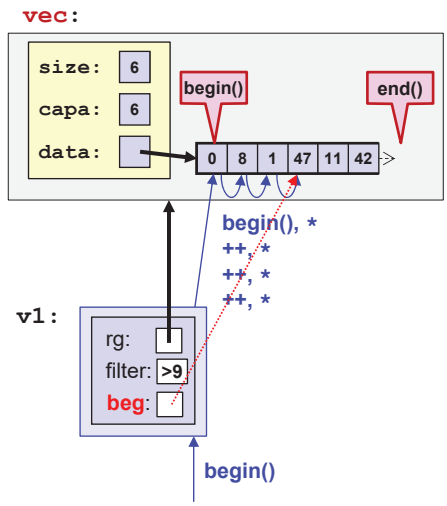
Multiple calls of **filter begin()**
One call of **filter end()**

In this scenario `begin()` grows quadratically


©2026 by josuttis.com

## Caching `begin()`

C++20

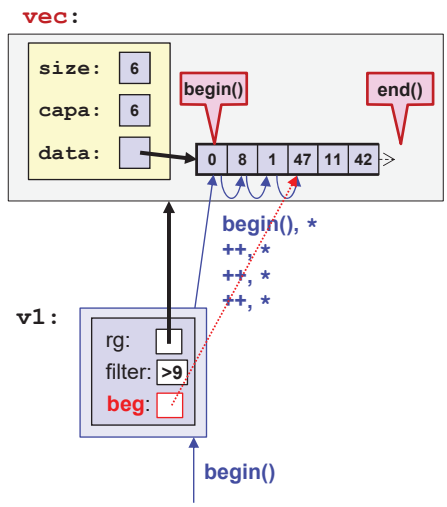



```
std::vector<int> vec{0, 8, 1, 47, 11, 42};
auto v1 = vec | std::views::filter(gt9);
auto pos = v1.begin();
```


©2026 by josuttis.com

## Caching `begin()`

C++20/C++23




```
std::vector<int> vec{0, 8, 1, 47, 11, 42};
auto v1 = vec | std::views::filter(gt9);
auto pos = v1.begin();
```

C++20 views that cache `begin()`:

Always: `filter` `drop_while` `split`

Sometimes: `drop` `reverse`

C++23 views that cache `begin()` or `end()`:

Always: `chunk_by`

Sometimes: `slide`


©2026 by josuttis.com

## Modifying Filtered Elements

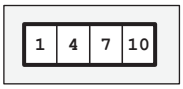
C++20

```
std::vector<int> coll{1, 4, 7, 10};
print(coll);

auto isEven = [] (auto&& i) { return i % 2 == 0; };
auto collEven = coll | std::views::filter(isEven);

// add 2 to even elements:
for (int& i : collEven) {
  i += 2;
}
print(coll);

// add 2 to even elements again:
for (int& i : collEven) {
  i += 2;
}
print(coll);
```




Output:

```
1 4 7 10
1 6 7 12
1 8 7 14
```



## Modifying Filtered Elements

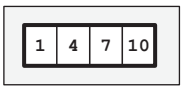
C++20

```
std::vector<int> coll{1, 4, 7, 10};
print(coll);

auto isEven = [] (auto&& i) { return i % 2 == 0; };
auto collEven = coll | std::views::filter(isEven);

// increment even elements:
for (int& i : collEven) {
  i += 1;          // OK
}
print(coll);

// increment even elements again:
for (int& i : collEven) {
  i += 1;          // Runtime Error: UB because predicate broken and applied again
}
print(coll);
```




Output:

```
1 4 7 10
1 5 7 11
1 6 7 11
```

## Modify Filtered Elements Ad Hoc

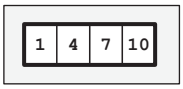
C++20

```
std::vector<int> coll{1, 4, 7, 10};
print(coll);

auto isEven = [] (auto&& i) { return i % 2 == 0; };


// increment even elements:
for (int& i : coll | std::views::filter(isEven)) {
  i += 1;          // OK
}
print(coll);

// increment even elements again:
for (int& i : coll | std::views::filter(isEven)) {
  i += 1;          // OK
}
print(coll);
```




Output:

```
1 4 7 10
1 5 7 11
1 5 7 11
```

Apply views ad hoc



## Modify Filtered Elements Ad Hoc

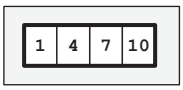
C++20

```
std::vector<int> coll{1, 4, 7, 10};
print(coll);

auto isEven = [] (auto&& i) { return i % 2 == 0; };
auto first3Even = std::views::filter(isEven) | std::views::take(3);

// increment even elements:
for (int& i : coll | first3Even) {
  i += 1;          // OK
}
print(coll);

// increment even elements again:
for (int& i : coll | first3Even) {
  i += 1;          // OK
}
print(coll);
```




OK to **pre-define** a view/pipeline **without** the **underlying range**

Output:

```
1 4 7 10
1 5 7 11
1 5 7 11
```

Apply views ad hoc

## Caching Views

C++20/C++23

```
std::vector<int> coll{1, 4, 7, 10};

// define view for even elements of coll:
auto even = [](int i){ return i % 2 == 0; };
auto collEven = coll |
                std::views::filter(even);
...


// modify underlying range:
coll[0] = 0;

// print collection and view:
std::println("coll: {}", coll);
std::println("even: {}", collEven);
```

Output:

```
coll: [0, 4, 7, 10]
even: [0, 4, 10]
```

```
std::vector<int> coll{1, 4, 7, 10};

// define view for even elements of coll:
auto even = [](int i){ return i % 2 == 0; };
auto collEven = coll |
                std::views::filter(even);
...
if (collEven.empty()) return;

// modify underlying range:
coll[0] = 0;

// print collection and view:
std::println("coll: {}", coll);
std::println("even: {}", collEven);
```

empty() check might change program behavior

Output:

```
coll: [0, 4, 7, 10]
even: [4, 10]
```



## Caching Views

C++20/C++23

```
std::vector<int> coll{1, 4, 7, 10};

// define view for even elements of coll:
auto even = [](int i){ return i % 2 == 0; };
auto collEven = coll |
                std::views::filter(even);
...


// modify underlying range:
coll[0] = 0;

// print collection and view:
std::println("coll: {}", coll);
std::println("even: {}", collEven);
```

Output:

```
coll: [0, 4, 7, 10]
even: [0, 4, 10]
```

```
std::vector<int> coll{1, 4, 7, 10};

// define view for even elements of coll:
auto even = [](int i){ return i % 2 == 0; };
auto collEven = coll |
                std::views::filter(even);
...
std::println("even: {}", collEven);

// modify underlying range:
coll[0] = 0;

// print collection and view:
std::println("coll: {}", coll);
std::println("even: {}", collEven);
```

debug output might change program behavior

Output:

```
even: [4, 10]
coll: [0, 4, 7, 10]
even: [4, 10]
```

## How to Deal with Caching Views

C++20/C++23

```
std::vector<int> coll{1, 4, 7, 10};

// define view for even elements of coll:
auto even = [](int i){ return i % 2 == 0; };
auto vEven = std::views::filter(even);
...


// modify underlying range:
coll[0] = 0;

// print collection and view:
std::println("coll: {}", coll);
std::println("even: {}", coll | vEven);
```

Apply views ad hoc

Output:

```
coll: [0, 4, 7, 10]
even: [0, 4, 10]
```

```
std::vector<int> coll{1, 4, 7, 10};

// define view for even elements of coll:
auto even = [](int i){ return i % 2 == 0; };
auto vEven = std::views::filter(even);
...
if ((coll | vEven).empty()) return;
std::println("even: {}", coll | vEven);

// modify underlying range:
coll[0] = 0;

// print collection and view:
std::println("coll: {}", coll);
std::println("even: {}", coll | vEven);
```

Apply views ad hoc

Output:

```
even: [4, 10]
coll: [0, 4, 7, 10]
even: [0, 4, 10]
```

**Note:**
Performance might suffer


©2026 by josuttis.com
https://www.godbolt.org/z/v54EEazdK

## C++ Views and Pipelines

## `const` Views


©2026 by josuttis.com

## C++20: Views

C++20

```
void print(const std::ranges::input_range auto& coll)
{
  for (const auto& elem : coll) {
    std::cout << elem << ' ';
  }
  std::cout << '\n';
}
```

Output:

```
0 8 15 7 11 42 2
0 2 7 8 11 15 42

0 8 15
0 2 7

0 8 15
0 2 7

0s 2s 7s
```

```
std::vector<int> coll1{0, 8, 15, 7, 11, 42, 2};
std::set<int> coll2{0, 8, 15, 7, 11, 42, 2};

print(coll1);
print(coll2);

print(std::views::take(coll1, 3));       // print first three elements
print(std::views::take(coll2, 3));       // print first three elements

print(coll1 | std::views::take(3));      // print first three elements
print(coll2 | std::views::take(3));      // print first three elements

print(coll2 | std::views::take(3)
            | std::views::transform([](auto v){
                                      return std::to_string(v) + 's';
                                    }));
```


©2026 by josuttis.com
https://www.godbolt.org/z/h1fKPvqTb

## Using Views

C++20

```
void print(const std::ranges::input_range auto& coll)
{
  for (const auto& elem : coll) {
    std::cout << elem << ' ';
  }
  std::cout << '\n';
}
```

Output:

```
0 8 15
0 2 7

7 11 42 2
ERROR

ERROR
7 11 2
```

```
std::vector<int> vec{0, 8, 15, 7, 11, 42, 2};
std::set<int> set{0, 8, 15, 7, 11, 42, 2};

print(vec | std::views::take(3));                                  // OK
print(set | std::views::take(3));                                  // OK

print(vec | std::views::drop(3));                                  // OK
print(set | std::views::drop(3));                                  // ERROR

print(vec | std::views::filter(isPrime));                          // ERROR

for (const auto& elem : vec | std::views::filter(isPrime)) {       // OK
  std::cout << elem << ' ';
}
```


©2026 by josuttis.com
https://www.godbolt.org/z/KfPj5WYE3

## Using Views

C++23

```
void print(const std::ranges::input_range auto& coll)
{
  for (const auto& elem : coll) {
    std::print("{} ", elem);
  }
  std::println();
}
```

Output:

```
0 8 15
0 2 7
7 11 42 2
[0, 8, 15, 7] [42, 2]
[7] [42, 2]
ERROR
```

```
std::vector<int> vec{0, 8, 15, 7, 11, 42, 2};
std::set<int> set{0, 8, 15, 7, 11, 42, 2};

print(vec | std::views::take(3));                                          // OK
print(set | std::views::take(3));                                          // OK

print(vec | std::views::drop(3));                                          // OK

print(vec | std::views::lazy_split(11));                                   // OK

print(vec | std::views::drop(3) | std::views::lazy_split(11));             // OK
print(vec | std::views::lazy_split(11) | std::views::drop(3));             // ERROR
```



## Processing Containers and Views

C++20

```
void print(const auto& coll)
{
  for (const auto& elem : coll) {
    std::cout << elem << ' ';
  }
  std::cout << '\n';
}
```

Output:

```
0 8 15 7 11 42 2
0 8 15
-7 -11 -42 -2
ERROR
7 11 2
```

```
std::vector<int> vec{0, 8, 15, 7, 11, 42, 2};
print(vec);

print(vec | std::views::take(3));                                          // OK

print(vec | std::views::drop(3)                                            // OK
          | std::views::transform(std::negate{}));

print(vec | std::views::filter(isPrime));                                  // Compile-time ERROR

for (int v : vec | std::views::filter(isPrime)) {                          // OK
  std::cout << v << ' ';
}
```

Caching views cannot be used **const** because calling **begin()** (or **empty()**) is a **modification**

## Processing Containers and Views

C++20

```
void print(auto& coll)
{
  for (const auto& elem : coll) {
    std::cout << elem << ' ';
  }
  std::cout << '\n';
}
```

Output:

```
0 8 15 7 11 42 2
ERROR
ERROR
ERROR
7 11 2
```

```
std::vector<int> vec{0, 8, 15, 7, 11, 42, 2};
print(vec);

print(vec | std::views::take(3));

print(vec | std::views::drop(3)                          // Compile-time ERROR
          | std::views::transform(std::negate{}));       // Compile-time ERROR

print(vec | std::views::filter(isPrime));                // Compile-time ERROR

for (int v : vec | std::views::filter(isPrime)) {        // OK
  std::cout << v << ' ';
}
```

"cannot bind a **non-const lvalue reference to** an **rvalue** (temporary object)"



## Processing Containers and Views by Universal Reference

C++20

```
void print(auto&& coll)
{
  for (const auto& elem : coll) {
    std::cout << elem << ' ';
  }
  std::cout << '\n';
}
```

***Universal reference***

- Can universally refer **to lvalue** (named object) and **rvalue** (temporary object) without making it **const**

Output:

```
0 8 15 7 11 42 2
0 8 15
-7 -11 -42 -2
7 11 2
7 11 2
```

```
std::vector<int> vec{0, 8, 15, 7, 11, 42, 2};
print(vec);

print(vec | std::views::take(3));                        // OK

print(vec | std::views::drop(3)                          // OK
          | std::views::transform(std::negate{}));

print(vec | std::views::filter(isPrime));                // OK

for (int v : vec | std::views::filter(isPrime)) {        // OK
  std::cout << v << ' ';
}
```

## Printing Containers and Views with `std::println()` C++23

Output:

```
[0, 8, 15, 7, 11, 42, 2]
[0, 8, 15]
[-7, -11, -42, -2]
[7, 11, 2]
[[8, 15, 7], [42, 2]]
[[42, 2]]
```

```
std::vector<int> vec{0, 8, 15, 7, 11, 42, 2};
std::println("{}", vec);

std::println("{}", vec | std::views::take(3));                          // OK

std::println("{}", vec | std::views::drop(3)                            // OK
                       | std::views::transform(std::negate{}));

std::println("{}", vec | std::views::filter(isPrime));                  // OK

std::println("{}", vec | std::views::drop(1)                            // OK
                       | std::views::lazy_split(11));

std::println("{}", vec | std::views::lazy_split(11)                     // OK
                       | std::views::drop(1));
```


©2026 by josuttis.com


## Printing Containers and Views with `std::println()` C++23

```
namespace std {
 void print(formatstring fmt, auto&&... args);
 void println(formatstring fmt, auto&&... args);
}
```

`const auto&`
no longer works for read access
Universal references required
**=> Views compromise `const` correctness**

Output:

```
[0, 8, 15, 7, 11, 42, 2]
[0, 8, 15]
[-7, -11, -42, -2]
[7, 11, 2]
[[8, 15, 7], [42, 2]]
[[42, 2]]
```

```
std::vector<int> vec{0, 8, 15, 7, 11, 42, 2};
std::println("{}", vec);

std::println("{}", vec | std::views::take(3));                          // OK

std::println("{}", vec | std::views::drop(3)                            // OK
                       | std::views::transform(std::negate{}));

std::println("{}", vec | std::views::filter(isPrime));                  // OK

std::println("{}", vec | std::views::drop(1)                            // OK
                       | std::views::lazy_split(11));

std::println("{}", vec | std::views::lazy_split(11)                     // OK
                       | std::views::drop(1));
```


©2026 by josuttis.com

# C++ Views and Pipelines

## `const` Propagation


©2026 by josuttis.com

## Sorting Multiple Ranges with the Zip View

C++23

```
int main()
{
  std::vector<std::string> first{"Brad", "Tina", "James"};
  std::deque<std::string> last{"Pitt", "Turner", "Bond"};

  std::println("first: {}", first);
  std::println("last:  {}\n", last);

  // use zip view to sort two ranges according to second range:
  std::ranges::sort(std::views::zip(first, last),          // let elems refer to both ranges
                    [] (auto tpl1, auto tpl2) {            // compare acc. to 2nd range
                      return std::get<1>(tpl1) < std::get<1>(tpl2);
                    });

  std::println("first: {}", first);
  std::println("last:  {}", last);
}
```

Output:

```
first: ["Brad", "Tina", "James"]
last:  ["Pitt", "Turner", "Bond"]

first: ["James", "Brad", "Tina"]
last:  ["Bond", "Pitt", "Turner"]
```


©2026 by josuttis.com
https://www.godbolt.org/z/z7G895a8o

## Elements with References Semantics

C++23

- **When elements have reference semantics**
  - Declaring them `const` does not disable modifications
  - Declaring them as a references does not improve performance

```
std::vector<std::string> vec{"tic", "tac", "toe"};
...
for (const auto& elem : vec | std::views::enumerate) {    // elements are tuples
  std::get<1>(elem)[0] = 'T';      // OOPS: modifies referred element
}
...
for (const auto& [idx, value] : vec | std::views::enumerate) {
  value[0] = 't';                  // OOPS: modifies referred element
}
...
for (auto [idx, value] : vec | std::views::enumerate) {
  value[0] = 'T';                  // OOPS: modifies referred element
}
...
```


©2026 by josuttis.com
https://www.godbolt.org/z/KohbrKcP9

## Elements with References Semantics

C++23

- **When elements have reference semantics**
  - Declaring them `const` does not disable modifications
  - View `as_const` to make elements really `const`

```
std::vector<std::string> vec{"tic", "tac", "toe"};
...
for (auto elem : vec | std::views::enumerate | std::views::as_const) {
  std::get<1>(elem)[0] = 'T';      // OK: ERROR
}
...
for (auto [idx, value] : vec | std::views::enumerate | std::views::as_const) {
  value[0] = 't';                  // OK: ERROR
}
...
```

**Use the as_const view**
instead of declaring elements `const`


©2026 by josuttis.com
https://www.godbolt.org/z/z9WEsebbo

## C++ Views and Pipelines

# The Filter View Fiasco


©2026 by josuttis.com

## The Filter View Fiasco 1/4

C++23

```
#include <print>
#include <string>
#include <array>
#include <vector>
#include <ranges>

int main()
{
  std::array<std::string, 4> coll{"Amsterdam", "Rome", "Cologne", "NY"};
  std::println("coll: {}\n", coll);

  // move short strings in reverse order to another container:
  auto small = [](const auto& s) { return s.size() <= 5; };
  auto sub = coll | std::views::filter(small)          // short strings of coll
                  | std::views::reverse                // in reverse order
                  | std::views::as_rvalue              // move them
                  | std::ranges::to<std::vector>();    // into a vector

  std::println("coll: {}", coll);
  std::println("sub:  {}", sub);
}
```

Output:

```
coll: ["Amsterdam", "Rome", "Cologne", "NY"]

coll: ["Amsterdam", "", "Cologne", ""]
sub:  ["NY", "Rome"]
```


©2026 by josuttis.com
https://www.godbolt.org/z/1K35Ws19W

## The Filter View Fiasco 2/4

C++23

```
#include <print>
#include <string>
#include <array>
#include <vector>
#include <ranges>

int main()
{
  std::array<std::string, 4> coll{"Amsterdam", "Rome", "Cologne", "NY"};
  std::println("coll: {}\n", coll);

  // move long strings in reverse order to another container:
  auto large = [](const auto& s) { return s.size() > 5; };
  auto sub = coll | std::views::filter(large)          // long strings of coll
                  | std::views::reverse                // in reverse order
                  | std::views::as_rvalue              // move them
                  | std::ranges::to<std::vector>();    // into a vector

  std::println("coll: {}", coll);
  std::println("sub:  {}", sub);
}
```

Undefined behavior
Because
- filter modifies (due to move)
- breaking its predicate

predicate inverted

Output:

```
coll: ["Amsterdam", "Rome", "Cologne", "NY"]

coll: ["Amsterdam", "Rome", "", "NY"]
sub:  ["Cologne"]
```


©2026 by josuttis.com
https://www.godbolt.org/z/jxxKMEcYv

## The Filter View Fiasco 3/4

C++23

```
#include <print>
#include <string>
#include <array>
#include <vector>
#include <ranges>

int main()
{
  std::array<std::string, 4> coll{"Amsterdam", "Rome", "Cologne", "New York"};
  std::println("coll: {}\n", coll);

  // move long strings in reverse order to another container:
  auto large = [](const auto& s) { return s.size() > 5; };
  auto sub = coll | std::views::filter(large)          // long strings of coll
                  | std::views::reverse                // in reverse order
                  | std::views::as_rvalue              // move them
                  | std::ranges::to<std::vector>();    // into a vector

  std::println("coll: {}", coll);
  std::println("sub:  {}", sub);
}
```

Undefined behavior
Because
- filter modifies (due to move)
- breaking its predicate

different value

Output:

```
coll: ["Amsterdam", "Rome", "Cologne", "New York"]

FATAL ERROR: program terminated
```

Iterates beyond coll and moves memory of other objects


©2026 by josuttis.com
https://www.godbolt.org/z/M68jx1deT

## The Filter View Fiasco 4/4

C++23

```
...
#include <ranges>

int main()
{
  const std::array<std::string, 4> data{"Berlin", "Rio", "Bengaluru", "Sydney"};

  std::array<std::string, 4> coll{"Amsterdam", "Rome", "Cologne", "New York"};
  std::println("coll: {}\n", coll);

  // move four long strings in reverse order to another container:
  auto large = [](const auto& s) { return s.size() > 5; };
  auto sub = coll | std::views::filter(large)          // long strings of coll
                  | std::views::take(4)                // at most 4 elements
                  | std::views::reverse                // in reverse order
                  | std::views::as_rvalue              // move them
                  | std::ranges::to<std::vector>();    // into a vector

  std::println("coll: {}", coll);
  std::println("sub:  {}", sub);
  std::println("data: {}", data);
}
```

Undefined behavior
Because
- filter modifies (due to move)
- breaking its predicate

limit loop

Possible Output:

```
coll: ["Amsterdam", "Rome", "Cologne", "New York"]

coll: ["", "Rome", "Cologne", ""]
sub:  ["New York", "Amsterdam", "Bengaluru"]
data: ["Berlin", "Rio", "", "Sydney"]
```


©2026 by josuttis.com
https://www.godbolt.org/z/5aE9ejPvj

## C++ Views and Pipelines

## Summary


©2026 by josuttis.com

## Views Considered Harmful

C++20/C++23

- **Some combination of views unexpectedly are not valid**
- **Some views do not support pure read access with `const`**
  - Using `const&` might no longer work (const correctness is broken)
- **Adding read statements might change behavior**
  - A print statement might change behavior
  - Calling `empty()` might change behavior
- **For some views, declaring elements `const` has no effect**
- **Some views might overwrite memory of other objects**
  - Non-intuitively
  - Especially the filter view is broken (see "*The Filter View Fiasco*")
    - Use it without reverse and/or with an `as_input` view in front (C++26)

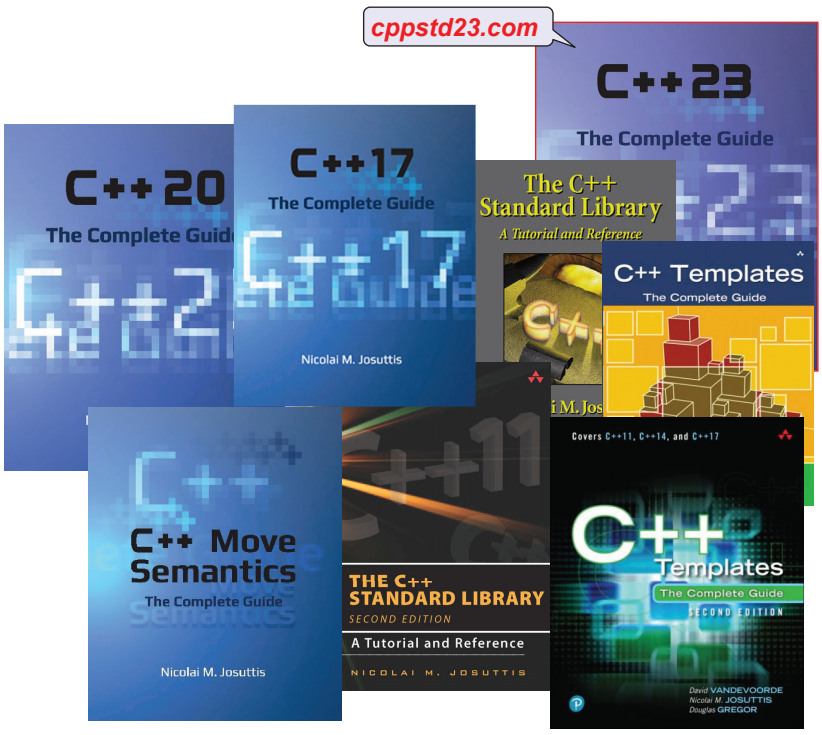

©2026 by josuttis.com

## Thank You!



**Nicolai M. Josuttis**

**www.josuttis.com**
**nico@josuttis.com**
**@NicoJosuttis**





©2026 by josuttis.com